23XT66 - CLOUD COMPUTING LAB / 23XT68 - SOFTWARE PATTERNS LAB

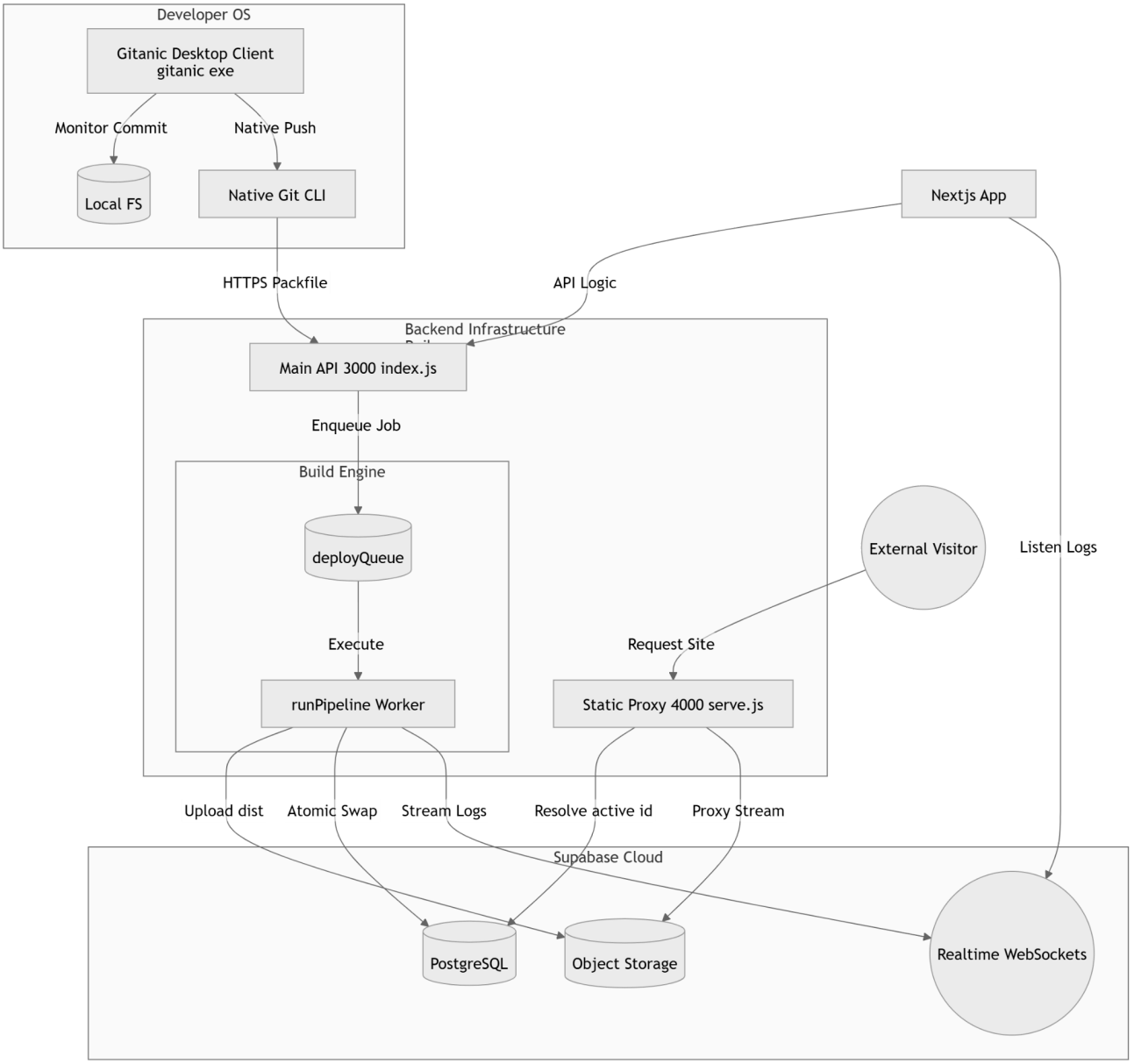
Gitanic

23PT01 - Aakash Velusamy | 23PT13 - Jovisha Curlie K

OVERVIEW

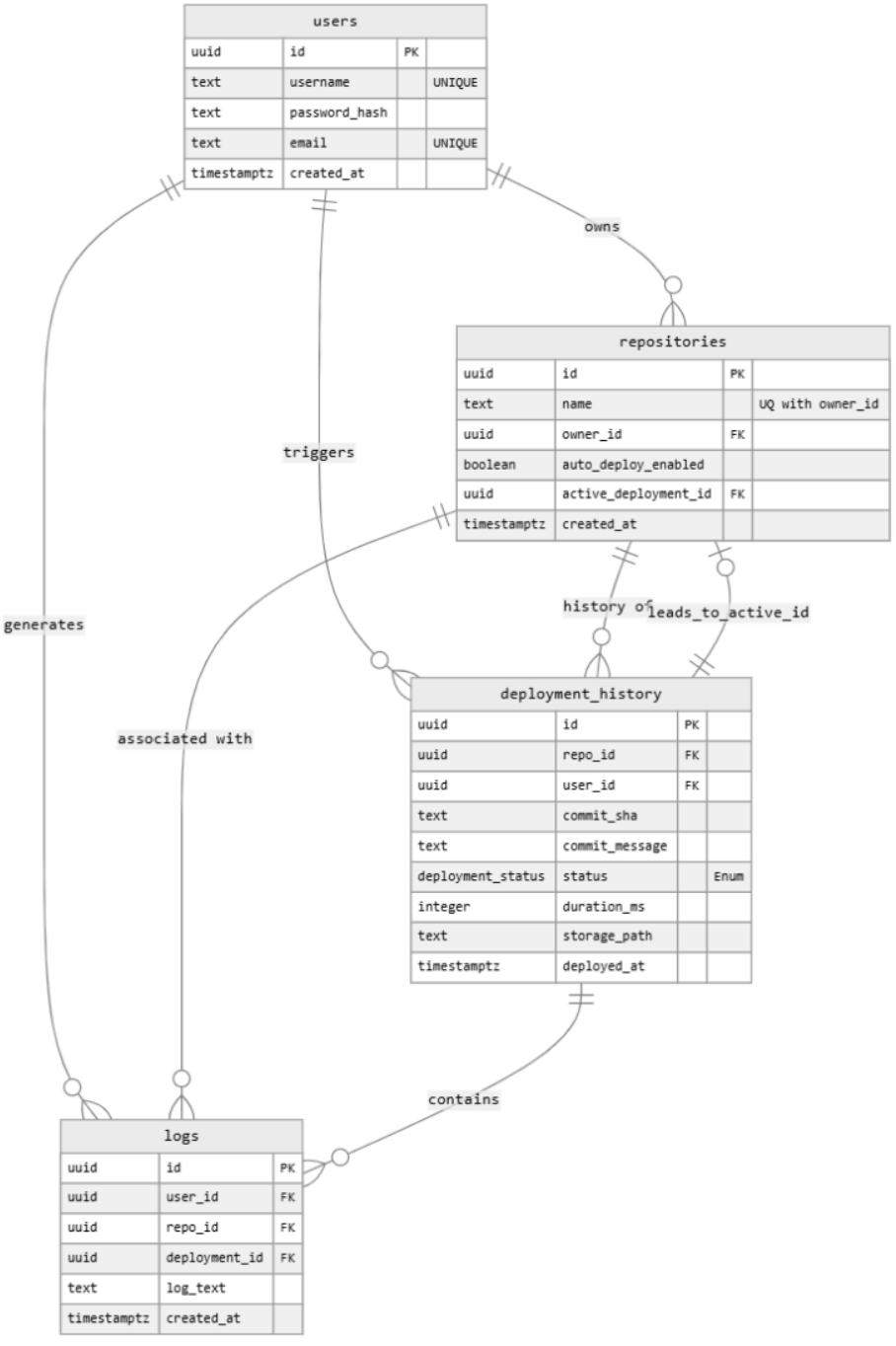
Gitanic is a distributed cloud-native Git hosting platform combined with a static website deployment system that allows developers to create repositories, push source code, and deploy static websites automatically through a unified interface. The platform also includes a desktop client that provides a graphical interface similar to GitHub Desktop, and the entire application is packaged as a single portable executable for easy distribution and usage.

SYSTEM ARCHITECTURE



Cloud Service	Primary Responsibility
Vercel	Hosts the frontend interface, while handling dynamic request resolution at runtime to serve deployed websites without relying on complex wildcard domain configuration.
Railway	Runs the core backend server inside a lightweight container environment, where it manages background job queues, executes deployment workflows, and stores repository data, allowing controlled and isolated execution of backend processes.
Supabase	Manages the database, enforces access rules for user-level isolation, handles object storage for production assets, and streams deployment logs in real time for monitoring.

ENTITY RELATIONSHIP



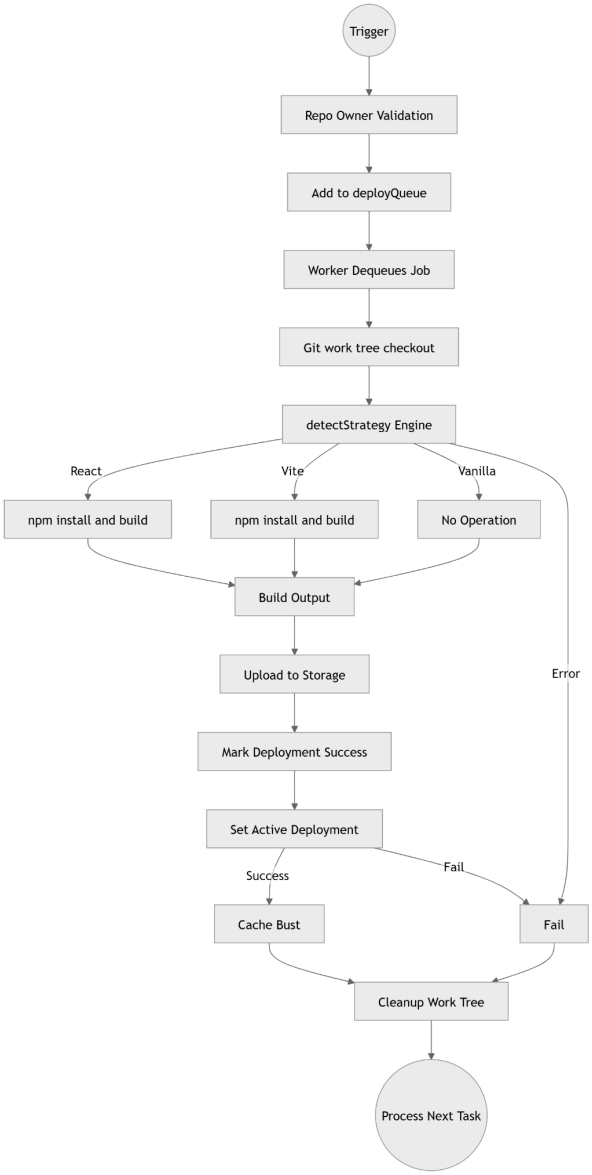
SOFTWARE PATTERNS USED

Pattern	Type	Application	Functionality
Layered Architecture	Architectural	Backend Structuring	Segregates the system into specialized tiers for request routing, business logic, and data storage. This maintains a clean separation of concerns and prevents infrastructure changes from affecting core logic.
Repository	Architectural	Data Persistence Layer	Abstracts data access through dedicated interfaces so that the application logic does not depend on the underlying database implementation, simplifying data management and security.
Strategy	Behavioral	Multi-Framework Build Engine	Orchestrates framework-specific build logic by dynamically selecting the appropriate execution path based on the project's structure, allowing the system to support a wide variety of web frameworks.
Observer	Behavioral	Real-time Communication	Monitors internal system events and triggers secondary actions like broadcasting live build updates to the user interface, ensuring the UI stays in sync with server-side processing.

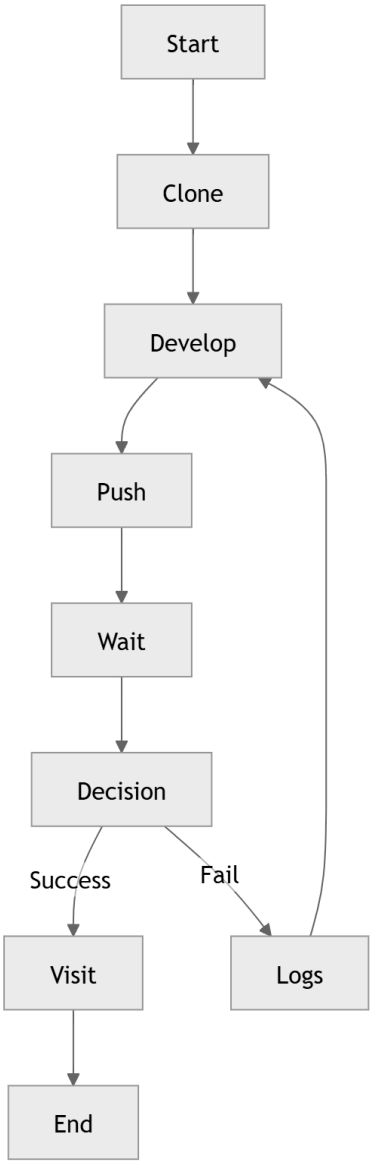
Producer-Consumer	Concurrency	Async Task Management	Manages resource-intensive deployment tasks by placing them in an ordered sequence. This ensures that the server processes heavy workloads predictably without overwhelming its physical resources.
Blue-Green Deployment	Deployment	Deployment Release Cycle	Minimizes deployment risk by building new versions in isolation and only updating the production pointer once the build is fully validated, ensuring zero downtime and safe rollbacks.
Singleton	Creational	Global State Management	Maintains single, consistent instances of shared system resources - such as background task schedulers and database connection pools - to prevent duplication and resource conflicts.
Proxy	Structural	Traffic Routing & Security	Acts as an intermediary that intercepts requests to either inject security credentials or route an incoming web request to the correct version of a hosted static site.
Facade	Structural	System Complexity Shield	Provides a simplified interface to the complex build and deployment subsystem, hiding the intricate details of version control and

			build environments from the rest of the application.
MVVM	Architectural	Frontend State Synchronization	Separates the user interface into independent units where the visual templates are driven by reactive data models, ensuring the view automatically reflects the current state of the application.
MVC	Architectural	Backend System Design	Separates the application into three logical components: the data structure, the response payload, and the control logic. This ensures that the way data is stored is independent of how it is requested or presented to the client.
Command	Behavioral	Task Encapsulation	Represents a discrete unit of work such as a build request as a self-contained object. This allows tasks to be saved, moved through a queue, and triggered only when the system is ready to execute them.

DEPLOYMENT PIPELINE



USER JOURNEY



When a developer pushes code or manually triggers a deployment, the platform executes a structured workflow:

Job Queueing: Places deployment requests in a sequential queue so that only one build runs at a time, which prevents resource contention and avoids overloading the server.

Workspace Preparation: Creates a fresh and isolated workspace for each deployment, ensuring that every build starts in a clean and reproducible environment.

Framework Analysis: Scans project configuration files to determine the required build strategy, and to prevent deployment of repositories containing unsupported frameworks earlier.

Sandboxed Execution: Executes the build inside a sandboxed environment with restricted permission, controlled resource access, and enforced time constraint to ensure safe execution.

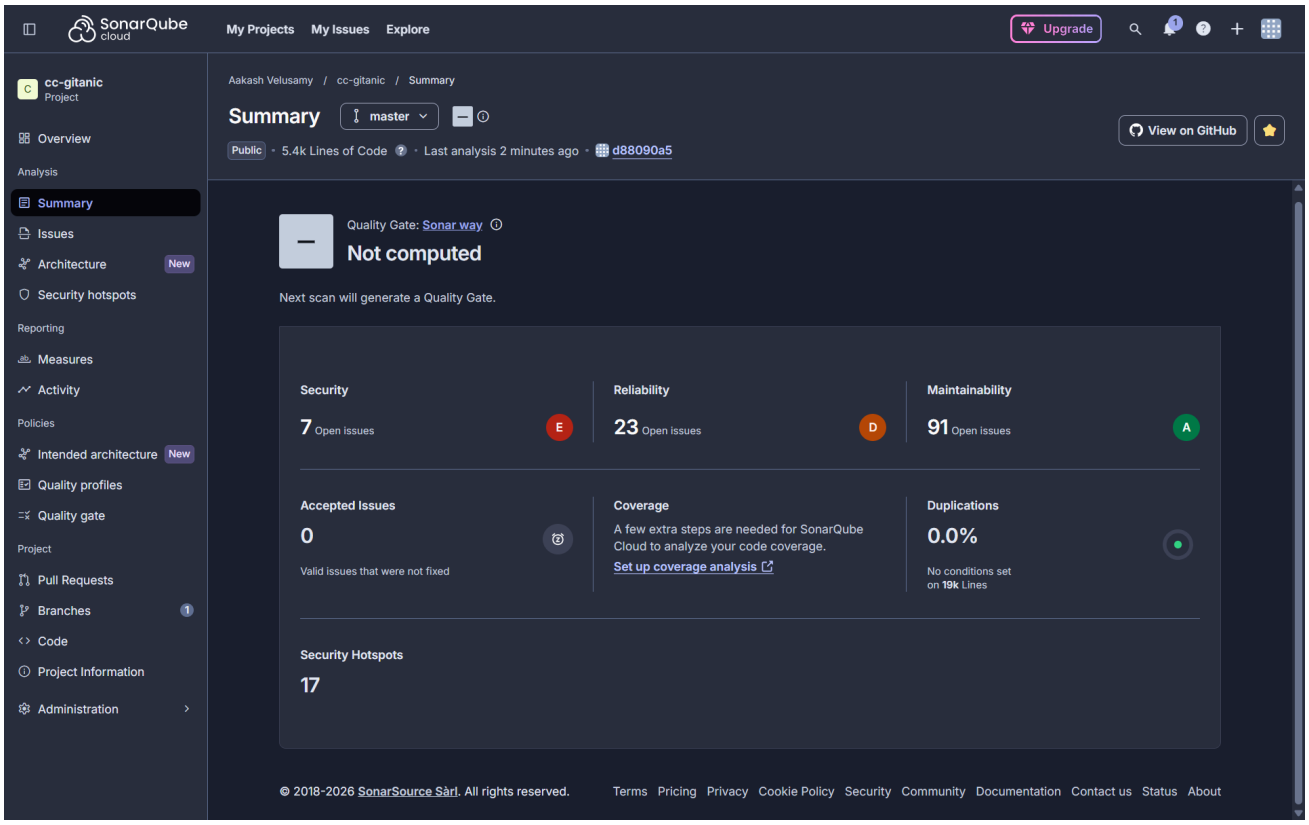
Parallel Object Upload: Splits generated files into chunks and uploads them concurrently, improving throughput and reducing overall deployment time.

Atomic Database Update: Updates the routing layer only after the deployment completes successfully, ensuring users are served a consistent and stable version.

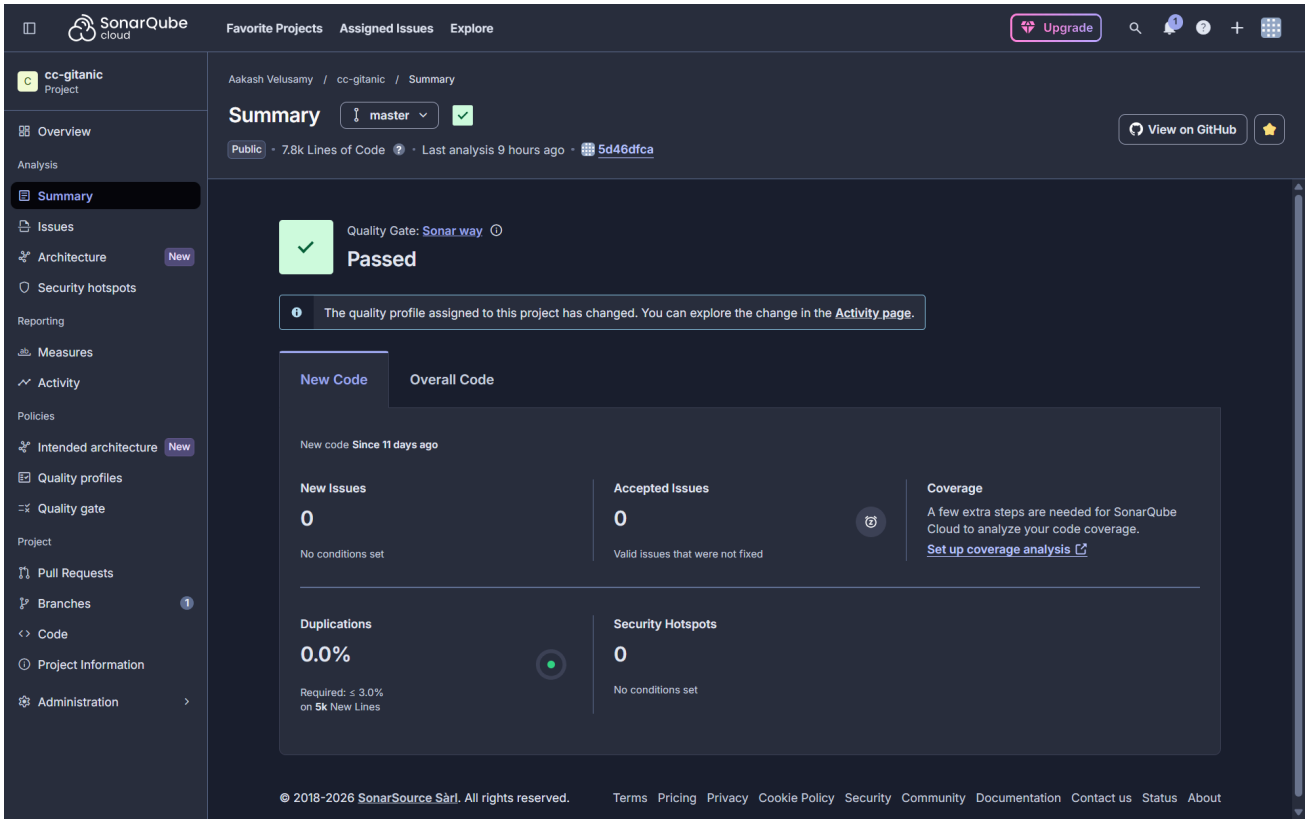
Workspace Reset: Resets the build environment after deployment, to free resources and maintain system hygiene.

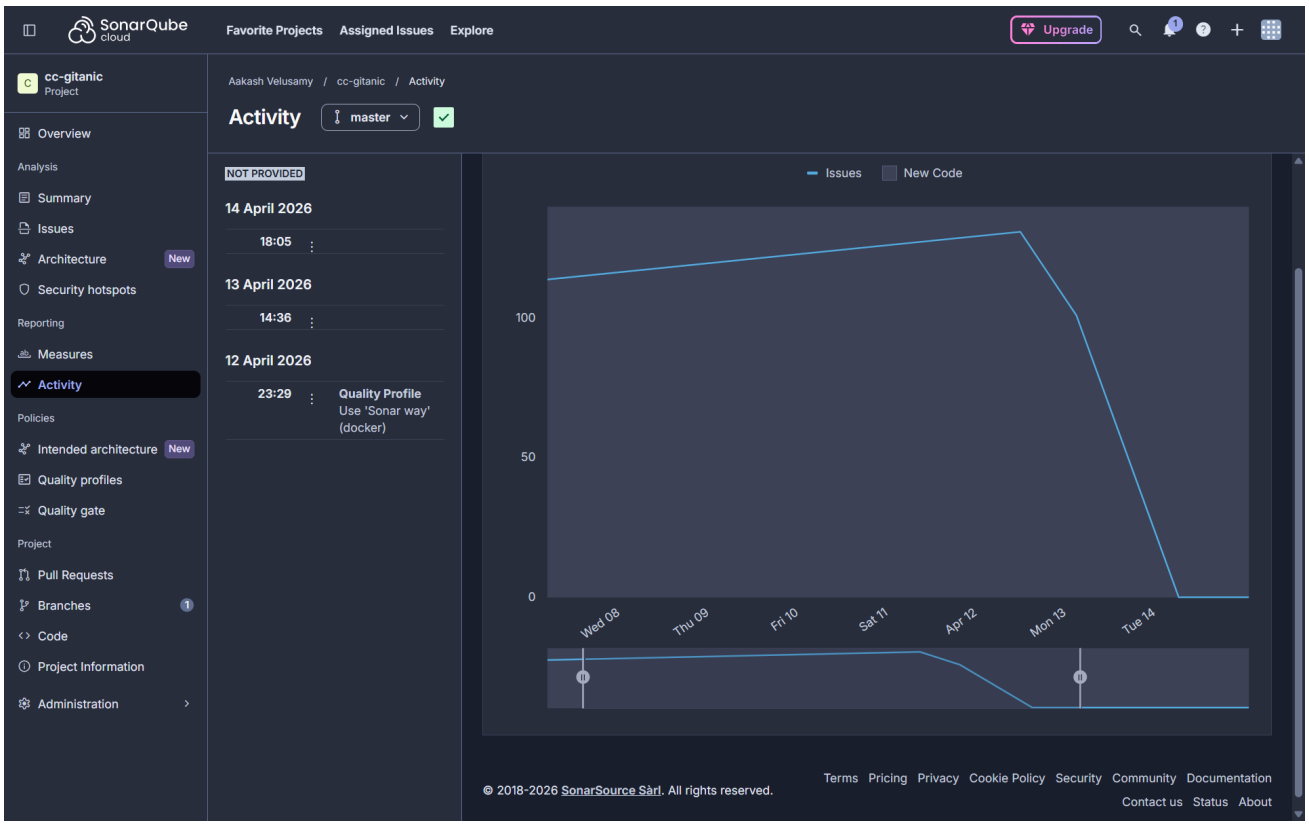
REFACTORING REPORT:

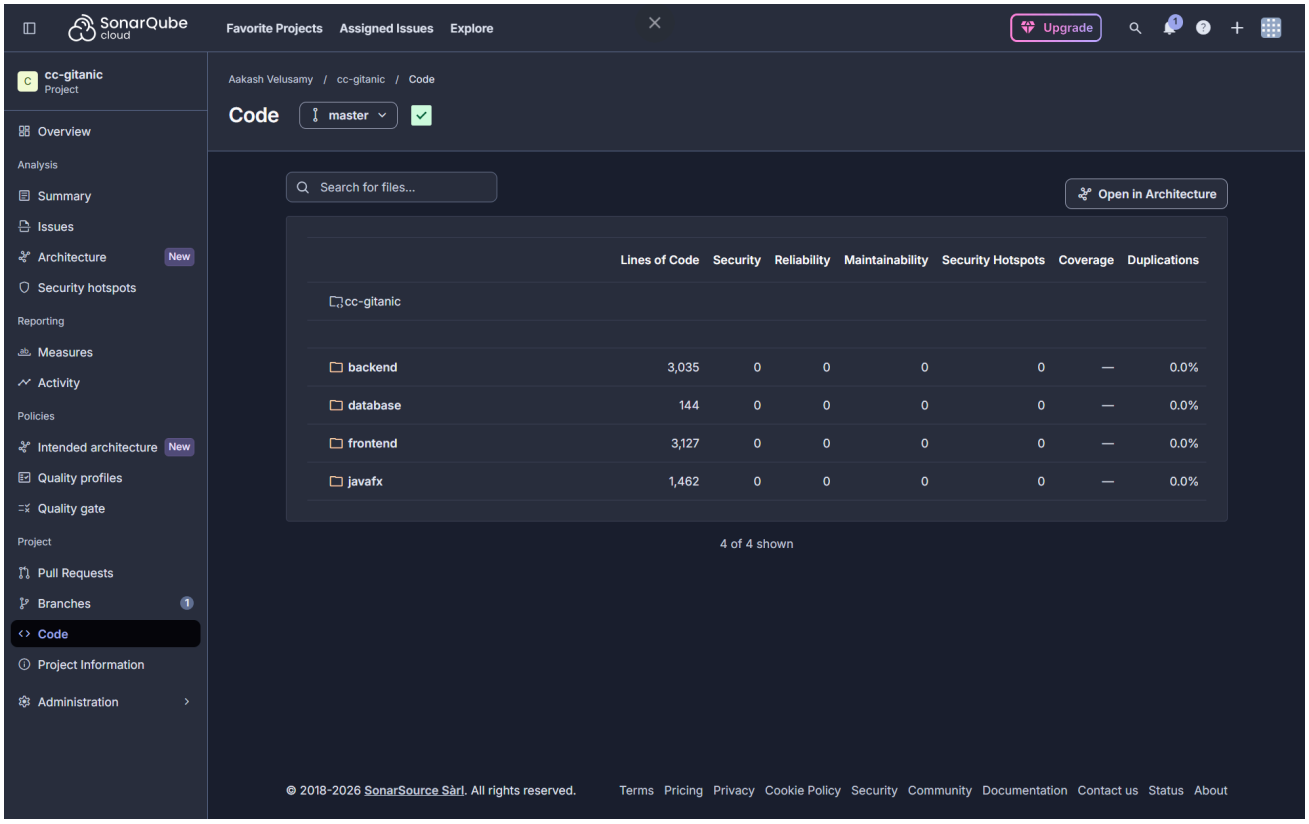
BEFORE:



AFTER:




SonarQube cloud
Favorite Projects
Assigned Issues
Explore
Upgrade
cc-gitanic
Project
Overview
Analysis
Summary
Issues
Architecture
New
Security hotspots
Reporting
Measures
Activity
Policies
Intended architecture
New
Quality profiles
Quality gate
Project
Pull Requests
Branches
Code
Project Information
Administration
Aakash Velusamy / cc-gitanic / Activity
Activity
master
NOT PROVIDED
14 April 2026
18:05
13 April 2026
14:36
12 April 2026
23:29
Quality Profile
Use 'Sonar way' (docker)
Issues
New Code
100
50
0
Wed 08
Thu 09
Fri 10
Sat 11
Apr 12
Mon 13
Tue 14
© 2018-2026 SonarSource Sàrl. All rights reserved.
Terms
Pricing
Privacy
Cookie Policy
Security
Community
Documentation
Contact us
Status
About


SonarQube cloud
Favorite Projects
Assigned Issues
Explore
Upgrade
cc-gitanic
Project
Overview
Analysis
Summary
Issues
Architecture
New
Security hotspots
Reporting
Measures
Activity
Policies
Intended architecture
New
Quality profiles
Quality gate
Project
Pull Requests
Branches
Code
Project Information
Administration
Aakash Velusamy / cc-gitanic / Code
Code
master
Search for files...
Open in Architecture
Lines of Code
Security
Reliability
Maintainability
Security Hotspots
Coverage
Duplications
cc-gitanic
backend 3,035 0 0 0 0 — 0.0%
database 144 0 0 0 0 — 0.0%
frontend 3,127 0 0 0 0 — 0.0%
javafx 1,462 0 0 0 0 — 0.0%
4 of 4 shown
© 2018-2026 SonarSource Sàrl. All rights reserved.
Terms
Pricing
Privacy
Cookie Policy
Security
Community
Documentation
Contact us
Status
About

TECHNOLOGY STACK

Category	Stack Used
Frontend	Next.js + Tailwind CSS
Backend	Node.js + Express.js
Database	PostgreSQL
Version Control	Git - CLI + Smart HTTP
Deployment	Linux - child_process + npm
CI/CD	Git Push Trigger
Desktop Application	JavaFX
Authentication	JWT + Basic Auth

DEPLOYMENT INFRASTRUCTURE

Layer	Provider
Frontend	Vercel
Backend	Railway
Database	Supabase
Object Storage	Supabase
Routing	Vercel Wildcard Domain
Logging	Supabase Realtime

CONCLUSION

Overall, this project helped in understanding how to design a system by dividing it into components with a separate responsibility, while also highlighting through the deployment flow the importance of controlled execution, proper sequencing, and isolation for maintaining reliable behavior, and strengthening the ability to think at a system level and design an application that remain stable and maintainable over time.